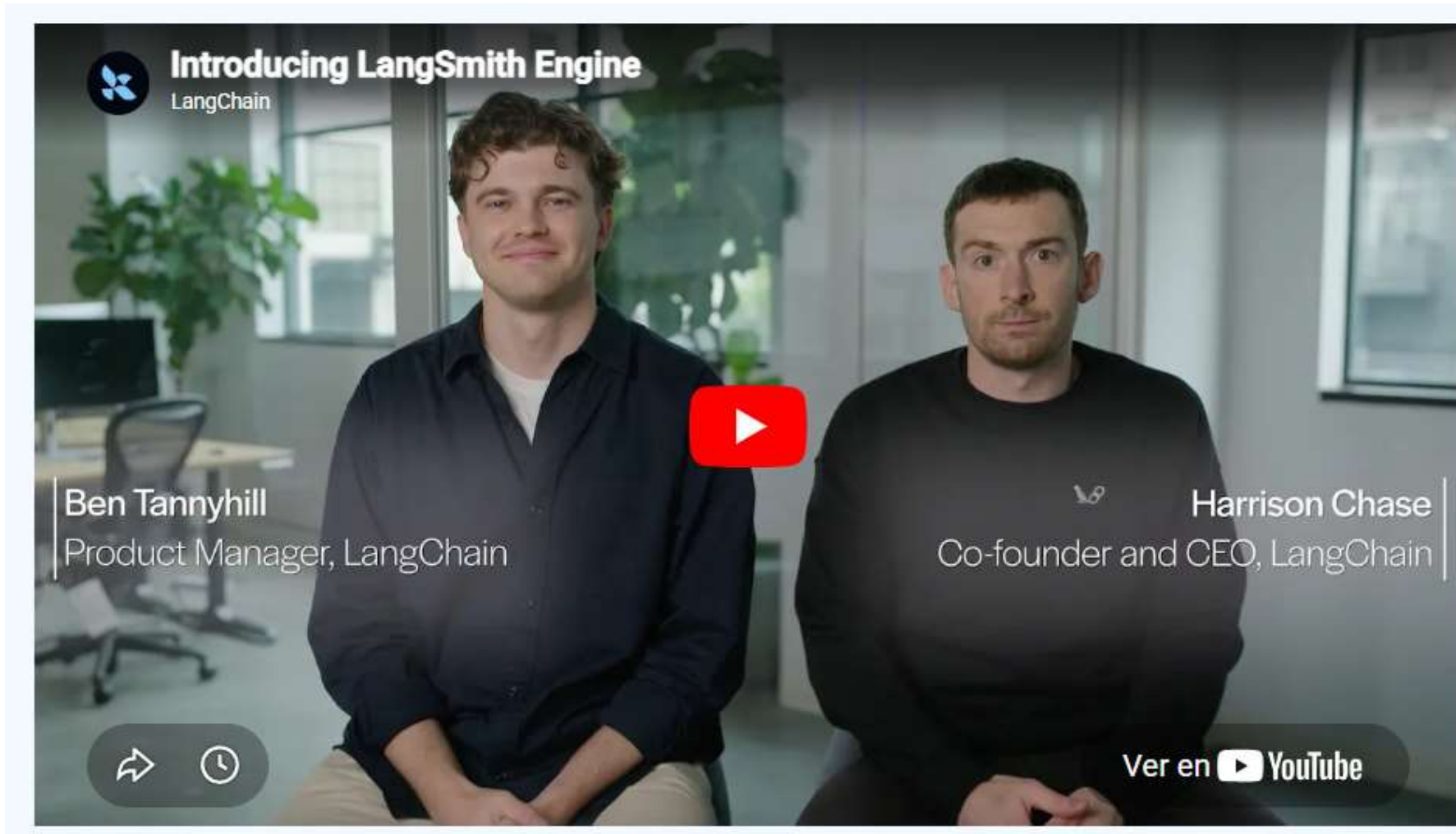


Ingeniería de soluciones con AI

Semestre 2026-1

Docente:
Francisco Macaya

LangSmith Engine: “Los agentes se ajustan solos”



Notas: <https://www.langchain.com/blog/introducing-langsmith-engine>

LangSmith Engine: ¿Cuáles son los patrones?

Ciclo de los agentes

Trace your agent to understand what it's doing

Identify patterns in failures or gaps in functionality

Make changes to prompts, tools, logic, or structure

Create ground truth datasets from production traces

Run experiments to confirm improvements and check for regressions and then ship and repeat

LangSmith Engine: ¿Cómo realmente luce?

The screenshot displays the LangSmith Engine interface for a workspace named 'chatbot-duoc-uc'. The left sidebar shows navigation options: Application, Search, Home, Tracing (17), Monitoring, Datasets & Experiments (1), Evaluators (1), Annotation Queues (1), Prompts, Playground, Studio, Context Hub, Deployments, and Sandboxes. The main area shows a list of issues under 'Tracing' for 'chatbot-duoc-uc'. The issues are:

- Tracing instrumentation gaps: orphan guardrail roots, null cost, no thread_id, default root name** (Tracing Quality, 3 traces, 45m ago)
- Agent fabricates course class counts under user pressure** (Hallucination, 4 traces, 45m ago)
- Agent loops rag_search 3-5x on unanswerable questions, retrying L...** (Failed Error Recovery, 4 traces, 45m ago)

The detailed view of the first issue shows a title 'Tracing instrumentation gaps: orphan guardrail roots, null cost, no thread_id, default root name' and a description: 'Four stacked LangSmith instrumentation gaps break core observability for this project: the three guardrail ChatOpenAI calls run in a raw ThreadPoolExecutor so they appear as orphan root runs (3 per turn, ~75% of the trace list), every LLM run is missing ls_provider / ls_model_name so total_cost is null project-wide, no thread_id is mirrored into root-run metadata so the Threads view cannot group a user's Telegram conversation, and the root run is named LangGraph (the framework default) so run rules and trace_filter queries cannot scope to this app. Cost dashboards, the Threads view, run-rule scoping, and trace-list signal-to-noise are all broken simultaneously.'

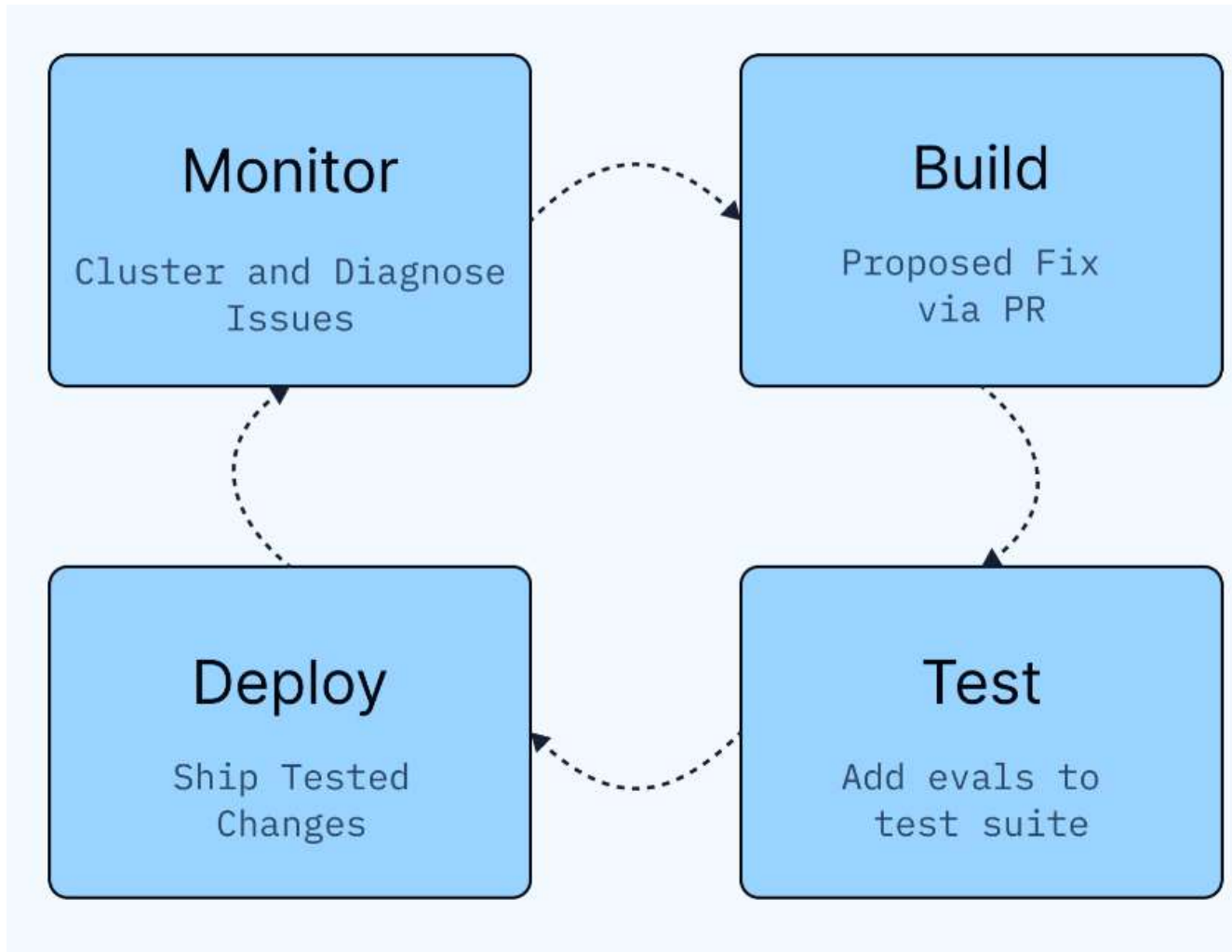
A timeline graph shows the issue's activity from 8 AM to 8 PM, with a peak around 1 PM. Below the graph, 'Linked Traces' are listed:

Trace ID	Description	Time	Cost
ChatOpenAI	Orphan ChatOpenAI guardrail root run that should be a child span of trace 019ed066-b7f3.	45m ago	1.77s
LangGraph	LangGraph root run with default name, no thread_id metadata, null cost; has 3 sibling orphan guardrail ChatOpenAI roots.	45m ago	5.03s
LangGraph	LangGraph root with ls_provider and ls_model_name missing on every child LLM span; total_cost null.	47m ago	15.82s

The 'Proposed Fix' section suggests: 'Close all four LangSmith tracing gaps in one commit: propagate the LangGraph run context into the guardrail thread pool so the three guardrail ChatOpenAI calls attach as child spans instead of orphan roots, pin langchain-openai>=0.3 so ls_provider / ls_model_name (and total_cost) are emitted on every LLM run, mirror the Telegram thread_id and chat_id into root-run metadata so the Threads view can group turns, and name the compiled graph asistente_clarita so run rules and trace_filter queries can scope to this app.'

At the bottom, a code diff shows changes to 'guardrails.py' with +14 lines and -11 lines.

LangSmith Engine: El ciclo se cierra.



Notas: <https://www.langchain.com/blog/introducing-langsmith-engine>

Cuesta al menos 50 dólares ...

*¿Que realmente hay detrás de **LangSmith Engine**?*

LangSmith Engine is powered by a **deep agent** that has access to your **trace data, evaluator feedback, and your agent's source code** (if connected to your repo).

It **monitors traces for several signal types**: explicit errors (tool call failures, timeouts), online evaluator failures, trace anomalies (latency spikes, token blowouts, unexpected step counts), negative user feedback, and unusual behaviors like users asking questions the agent wasn't built to answer. **When Engine spots a pattern across multiple traces, it clusters them into a single named issue rather than surfacing each failure individually.**

LangSmith Engine is built on top of LangSmith's existing tracing and evaluation infrastructure. It uses your existing evaluator results as inputs, so failures your evals catch feed directly into issue detection. When Engine proposes a new evaluator, it's because it detected a gap in your current coverage. When it creates a dataset example, it goes directly into your existing offline eval workflow.